

Overview

This report demonstrates how a downstream dependency can cause cascading failures in a distributed system and how a fail-fast timeout mitigates the issue.

The system consists of two microservices:

Search Service - Handles client HTTP requests and synchronously calls a downstream service.

Inventory Service - Simulates a dependency by introducing artificial latency and random failures.

Under load, the Search Service depends entirely on the Inventory Service to complete each request. When the downstream service becomes slow or unreliable, its latency propagates upstream, leading to request buildup, increased response times, and elevated error rates.

System Architecture

The system consists of two containerized microservices deployed on AWS ECS (Fargate):

- **Search Service (HTTP :8080)** – Exposed publicly through an Application Load Balancer (ALB). Handles incoming client requests.
- **Inventory Service (HTTP :8081)** – Internal downstream service accessed only by the Search Service.
- **Load Generator (Locust)** – Simulated 100 concurrent users with a spawn rate of 20 users per second targeting the ALB endpoint.

Architecture:

Client → ALB → Search Service → Inventory Service

For every incoming request, the Search Service makes a **synchronous HTTP call** to the Inventory Service before responding. Because the dependency is blocking, any latency or failure in the Inventory Service directly propagates upstream to the Search Service.

Failure Scenario (Problematic Deployment)

The Inventory Service was intentionally configured to simulate instability:

- Random response latency up to **3000ms**
- **30% probability of returning HTTP 5xx errors**

The Search Service was initially implemented without any resilience mechanisms:

- **No timeout** on downstream calls
- **No fallback response** when the dependency failed
- **Synchronous, blocking HTTP call** to Inventory for every request

Under load, this configuration caused the Search Service to wait for slow Inventory responses, resulting in request accumulation, increased latency, and elevated error rates across the system.

Observed Metrics (Broken System)

Load configuration:

- 100 concurrent users
- Spawn rate: 20 users per second

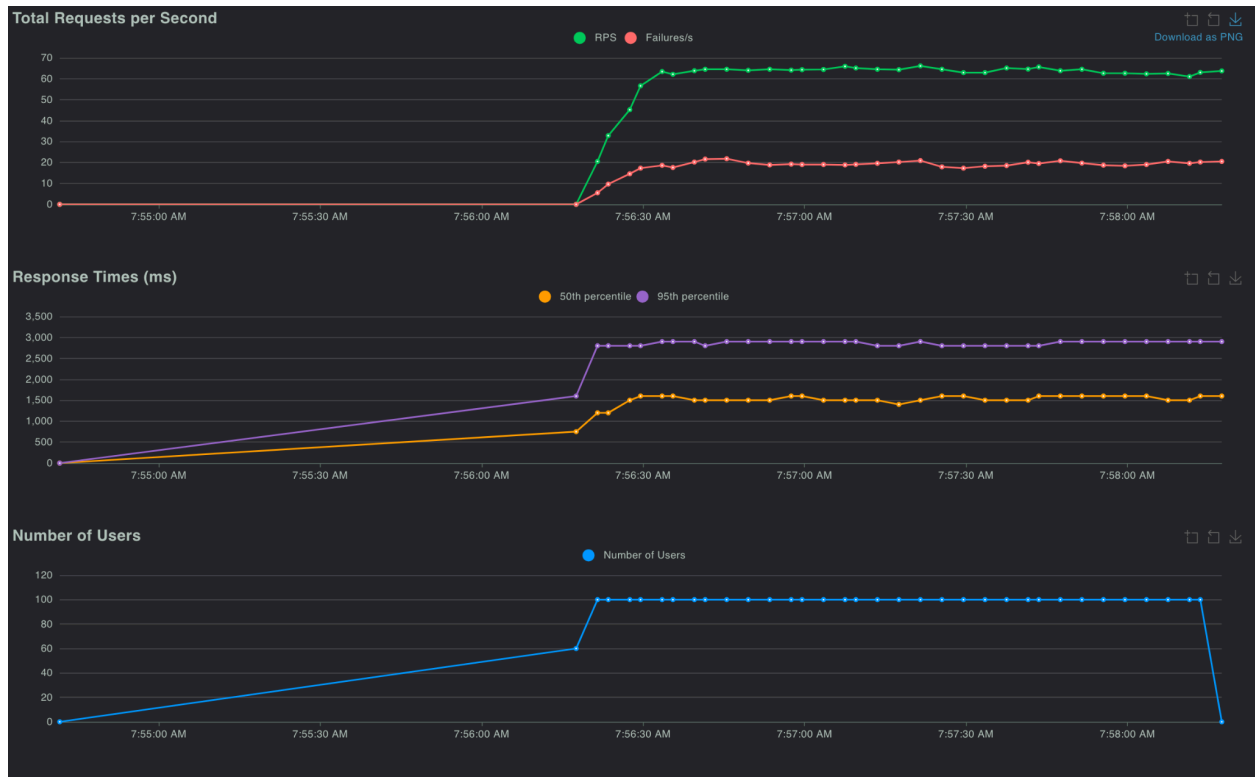
Observed results:

- **p95 latency:** $\approx 2800\text{--}3000\text{ms}$
- **Failures per second:** $\approx 15\text{--}20$
- **Throughput:** $\approx 60\text{--}65$ requests per second

Despite sustained load, throughput plateaued while latency increased significantly. The elevated p95 response time closely matched the maximum downstream delay, indicating that the Search Service was fully blocked on the Inventory dependency.

Conclusion:

Downstream latency propagated upstream, leading to request accumulation, latency amplification, and elevated error rates under load.



Root Cause

The Search Service performed a synchronous, blocking call to the Inventory Service without any timeout control. When the downstream dependency became slow or unreliable, each request remained blocked until completion or failure.

This led to the following failure chain:

Slow dependency → Thread blocking → Request queue buildup → Increased latency → 5xx errors

As concurrent requests accumulated, system resources became saturated, resulting in degraded throughput and elevated error rates. This behavior reflects a classic cascading failure pattern in distributed systems.

Resilience Strategy Implemented

To mitigate the issue, a **Fail-Fast pattern** was introduced.

The Search Service was updated to enforce a strict 200ms timeout on downstream calls. If the Inventory Service did not respond within this bound, the request was terminated early and a fallback response was returned instead of blocking indefinitely.

Implementation: `context.WithTimeout(200 * time.Millisecond)`

This change bounded latency, prevented request accumulation, and improved overall system stability under identical load conditions.

Improved System Results

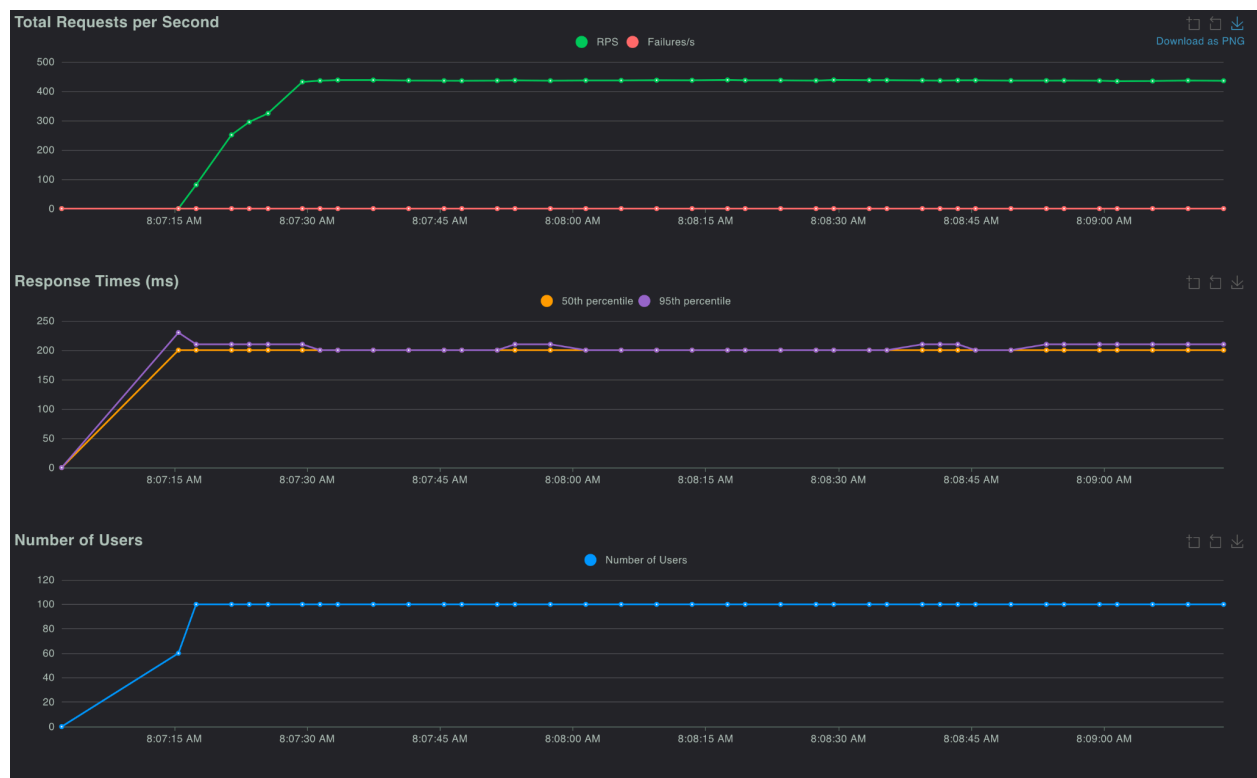
Load configuration remained unchanged:

- 100 concurrent users
- Spawn rate: 20 users per second

Observed results after implementing fail-fast timeout:

- **p95 latency:** $\approx 200\text{ms}$
- **Failures per second:** ≈ 0
- **Throughput:** $\approx 430+$ requests per second

Latency remained bounded near the configured timeout threshold, and the system maintained high throughput without error escalation. Under identical load conditions, the service remained responsive and stable.



Conclusion

In the initial deployment, the absence of timeout controls allowed downstream latency to propagate upstream, resulting in cascading performance degradation and elevated error rates.

By introducing a strict fail-fast timeout, downstream delays were contained, preventing request accumulation and resource saturation. This bounded latency, stabilized throughput, and significantly improved overall system resilience under load.